Product Evaluation Report

Hands-on Assessment of Dashboard, REST API, and Model Context Protocol (MCP)

Evaluator: Software Dev & Product Candidate **Date: November 12, 2026** **Duration: ~3.5 Hours Real Testing**
Scope: Dashboard, API v1, FastMCP **Environment: Windows / Chrome / Python** **API Key: 8aa4...d362 (Masked)**
Trial Subscription: API Premium (1,000,000 Credits)

1. Executive Summary

This evaluation provides a technical, evidence-based assessment of the FortyGuard developer ecosystem: the Temperature Dashboard, Temperature API (v1), and Temperature Model Context Protocol (MCP) server. Testing was performed against real production infrastructure using the provided trial API credentials. Overall, FortyGuard delivers impressive, high-value proprietary temperature models and atmospheric intelligence. The MCP server is the standout innovation, exposing clean, streamable JSON-RPC tools with real-time credit tracking. However, key friction points were verified: the official quickstart documentation provides unserved NYC coordinates that fail with OUT_OF_COVERAGE, the routing endpoint suffers from backend gateway timeouts (504), asynchronous failures lack diagnostic error messages, and the Heat Intelligence PDF reporting engine exhibits a unit-conversion mismatch where Celsius inputs are treated as Fahrenheit.

2. Product Scores & Summary

Product / Component	Score	Key Evidence & Observations
Temperature Dashboard	**3.5 / 5**	Polished login/auth UI, clean validation, but heavy tracking scripts and SPA deep link issues.
Temperature API (v1)	**3.8 / 5**	Accurate 100m GeoJSON tiles and environmental parameters; hurt by 504 timeouts and schema inco
Temperature MCP	**4.2 / 5**	Standout capability. Streamable HTTP / FastMCP, 10 live tools, fast credit balance queries (1.6s).
Overall Experience	**3.8 / 5**	Technically solid climate intelligence platform; ready for production with targeted polish.

3. Verified Technical Issues

Issue ID	Severity	Component	Description & Reproducibility
ISS-01	**P1 (High)**	API Documentation	Quickstart polygon coordinates (NYC: [-74.0170, 40.7050]) fail with 400 OUT_OF_COVERAGE
ISS-02	**P1 (High)**	API / Engine	POST /v1/routing consistently returns 504 Gateway Timeout from Envoy proxy under valid US c
ISS-03	**P2 (Medium)**	API Status Polling	Asynchronous failures (e.g. streetview, satellite) return status='Failed' without error message/rea
ISS-04	**P2 (Medium)**	Heat Intelligence	Unit mismatch: API input requires Celsius (35.0), but report displays 1.66°C (35.0°F) assuming
ISS-05	**P3 (Low)**	API Design	Inconsistent coordinate schema: sat.latitude/sat.longitude vs latitude/longitude vs origin.lat/lon.
ISS-06	**P3 (Low)**	MCP Documentation	Catalog documentation specifies 9 tools, but live FastMCP server exposes 10 tools (+credit_bal

4. Detailed Product Evaluations

A. FortyGuard Temperature Dashboard

- First Impression: Clean, modern enterprise SaaS aesthetic with cohesive indigo branding and typography.
- Form Validation: Submit button remains strictly disabled until valid email and password formats are entered.
- Error Feedback: Submitting non-existent credentials triggers an explicit PrimeNG toast alert: 'Error: Invalid email or password'.
- Network Overhead: Heavy third-party telemetry (Google Analytics 4, Mixpanel, and Microsoft Clarity) caused intermittent DOM timeouts.
- Client Routing: Navigating directly to /mcp/connect rendered an AI client pairing state ('No valid connection was found').

B. FortyGuard Temperature API (v1)

- Heatmap Engine: Submitted 1 km² polygon in Phoenix, AZ. Completed asynchronously in 24 seconds (8 polling cycles). Returned 2,835 lines of granular GeoJSON features with min/max/average temperatures and frequency distribution.
- Environmental Parameters: Returned 200 OK with apparent temperature (40.7°C), wet-bulb (23.7°C), relative humidity (22.9%), AQI (58.4), and clear-sky solar irradiance (GHI: 926.59 W/m²).
- Input Validation: Out-of-range latitude (>90) triggered 422 Unprocessable Entity with Pydantic field schemas. Non-US coordinates triggered 400 with code 'OUT_OF_COVERAGE'. Unclosed GeoJSON polygon rings triggered 400 Bad Request.
- Credit Accounting: Successfully verified that failed activities (streetview, satellite) deduct 0 credits, while completed heatmaps and env_params accurately deducted 4,220 and 2,900 credits respectively.

C. FortyGuard Temperature MCP Server

- Protocol Architecture: Remote Streamable HTTP / SSE implementation hosted at https://tos-mcp.app.fortyguard.com/mcp.
- Tool Discovery: Returned 10 fully specified FastMCP tools with rich input and output JSON schemas.
- Live Execution: Called credit_balance tool (1,611ms latency, 0 credits, returned 976,400 available credits) and env_params tool (successfully returned live thermal metrics).
- AI Integration: Highly practical for autonomous climate agents, urban planning assistants, and thermal risk workflows.

5. Top Prioritized Recommendations

1. Fix Quickstart Coordinates & Add Coverage Boundary Map *Priority: High | Documentation / API*

Replace the NYC polygon in quickstart with Phoenix, AZ coordinates and add a documented list or interactive map of supported cities to prevent immediate 400 errors for new developers.

2. Resolve Routing Endpoint 504 Timeout & Investigate Graph Solver *Priority: High | Routing Service*

Profile the OSRM/routing graph solver on POST /v1/routing to complete within Envoy proxy limits or return an asynchronous activity_id immediately like other endpoints.

3. Surface Diagnostic Failure Reasons in Status Polling Payload *Priority: Medium | API / Async Tasks*

Include error_code and error_message in status payload when an activity fails so developers know whether failures stem from missing imagery, invalid dates, or engine limits.

4. Correct Celsius/Fahrenheit Unit Pipeline in Heat Intelligence *Priority: Medium | Reporting Engine*

Ensure the PDF generation pipeline does not convert incoming Celsius values as Fahrenheit, preventing distorted climate reports.

5. Standardize Request Coordinate Schemas Across Endpoints *Priority: Low | API Design*

Unify coordinate inputs to a consistent standard (e.g., latitude/longitude at top-level) across satellite, routing, and environmental endpoints.